AI Customer Support Platform

Graduation Project

Final Submission: 28 September 2026
Team Size: 3–4 Students

---

1. Project Overview

The AI Customer Support Platform is a full-stack web application designed to help organizations manage customer support tickets while providing an AI-powered conversational assistant.

The platform allows customers to create and track support tickets, communicate with support agents, and interact with an AI chatbot.

The AI system should not act as a simple chatbot only. It should be implemented as an AI Agent capable of understanding user requests, deciding what action is required, and using available backend tools when necessary.

The project must demonstrate practical integration between:

- HTML
- CSS
- JavaScript
- Python
- FastAPI
- PostgreSQL
- LLM APIs
- LangChain
- LangGraph
- AI Agents
- Tool Calling
- n8n
- Git & GitHub

---

2. Project Objectives

The main objectives are:

1. Build a complete full-stack customer support system.
2. Implement a RESTful backend using FastAPI.
3. Store application data using PostgreSQL.
4. Implement authentication and role-based authorization.
5. Build a customer support ticket management system.
6. Build an AI-powered chatbot.
7. Implement an AI Agent using LangChain.
8. Implement agent workflow/state management using LangGraph.
9. Allow the AI Agent to interact with backend tools.
10. Integrate n8n for business automation.
11. Demonstrate professional Git/GitHub teamwork.
12. Deliver a working application within 12 days.

---

3. Scope

3.1 In Scope

The system must include:

- User authentication
- Role-based authorization
- Customer management
- Support agent management
- Ticket management
- Ticket status management
- Ticket priority
- Ticket conversations
- AI chatbot
- AI Agent
- LangChain integration
- LangGraph integration
- Tool Calling
- FastAPI tools for the Agent
- n8n automation
- Email/notification workflows
- Google Sheets integration
- Dashboard
- GitHub team workflow

3.2 Out of Scope

The following are NOT required:

- RAG
- Vector databases
- Embeddings
- Fine-tuning
- Complex payment systems

---

4. User Roles

The system must support at least three roles.

4.1 Customer

Customers can:

- Register
- Login
- View their profile
- Create tickets
- View their tickets
- View ticket details
- Add messages to tickets
- Track ticket status
- Track ticket priority
- Interact with the AI chatbot

4.2 Support Agent

Support agents can:

- Login
- View assigned tickets
- View customer information

- View ticket conversations
- Reply to customers
- Change ticket status
- Change ticket priority
- Assign/reassign tickets
- View AI-generated response suggestions
- Use the AI Assistant

4.3 Admin

Admins can:

- Login
- Manage users
- Manage support agents
- View all tickets
- Assign tickets
- Change ticket status
- Change ticket priority
- Delete/disable users
- View system statistics
- Monitor support activity
- View AI usage/activity
- Access administrative dashboard

---

5. Functional Requirements

FR-01 Authentication

The system shall provide:

- User registration
- User login
- Password hashing
- Authentication
- Logout
- Protected endpoints
- Role-based authorization

The backend should use token-based authentication such as JWT.

FR-02 User Management

Users shall have information such as:

- ID
- Name
- Email
- Password hash
- Role
- Created date
- Account status

FR-03 Ticket Creation

Customers shall be able to create support tickets.

Each ticket should contain:

- Ticket ID
- Customer
- Subject
- Description
- Category
- Priority
- Status
- Assigned support agent
- Created date
- Updated date

FR-04 Ticket Categories

The system should support categories such as:

- Technical Issue
- Account Issue

- Billing
- Product Issue
- General Inquiry

Teams may add additional categories.

FR-05 Ticket Priority

Tickets must support:

- Low
- Medium
- High
- Critical

FR-06 Ticket Status

Tickets must support:

- Open
- In Progress
- Waiting for Customer
- Resolved
- Closed

FR-07 Ticket Conversation

Each ticket should contain a conversation between the customer and support team.

Messages should include:

- Message ID
- Ticket ID
- Sender
- Message
- Timestamp

---

6. AI Chatbot

The application must include a chatbot interface.

The chatbot should allow users to communicate naturally with the AI system.

Example:

Customer:

My account is not working and I cannot login.

The chatbot should understand the request and provide an appropriate response.

The chatbot must be connected to the application's AI backend rather than being a static frontend feature.

---

7. AI Agent

The chatbot must use an AI Agent architecture.

The Agent should be capable of:

1. Understanding the user request.
2. Determining whether a tool is required.
3. Selecting an appropriate tool.
4. Executing the tool.
5. Processing the result.
6. Generating the final response.

The Agent must NOT simply generate a text response for every request.

---

8. Required Agent Tools

The team must implement backend tools that the Agent can call.

At least the following tools are required:

Tool 1 — Get Customer Tickets

The Agent can retrieve the customer's tickets.

Example request:

Show me my open tickets.

The Agent should call the appropriate FastAPI/backend functionality.

Tool 2 — Get Ticket Details

The Agent can retrieve information about a specific ticket.

Example:

What's the status of ticket #125?

Tool 3 — Create Ticket

The Agent can create a ticket when the user explicitly requests it.

Example:

I have a problem with my account. Create a support ticket for me.

Tool 4 — Check Ticket Status

The Agent can check the current status of a ticket.

Tool 5 — Update Ticket

The Agent may update permitted ticket information.

For example:

- Priority
- Status

Sensitive operations should require appropriate authorization.

Tool 6 — Escalate Ticket

The Agent can escalate a ticket when appropriate.

Example:

I still have the same problem after trying the suggested solution.

The Agent can trigger an escalation process.

---

9. LangChain Requirements

LangChain must be used as part of the AI implementation.

The team should demonstrate:

- LLM integration
- Prompt templates
- Messages
- Chat model
- Agent/tool integration
- Tool calling
- Structured interaction with backend tools

The project must clearly demonstrate that LangChain is actually being used in the application.

Simply importing LangChain without using it for the Agent is not acceptable.

---

10. LangGraph Requirements

LangGraph must be used to implement the Agent workflow.

The Agent workflow should contain multiple states/nodes.

A suggested workflow:

User Message
↓
Understand Request
↓
Determine Action
↓
Tool Required?
↓
Yes → Execute Tool
↓
Process Result
↓
Generate Response
↓
Return Response

The team should implement a meaningful graph rather than using LangGraph only as a wrapper.

---

11. Suggested LangGraph Workflow

A possible implementation:

Node 1 — Receive Message

Receives the user's message.

Node 2 — Analyze Request

Determines the user's intent.

Examples:

- Ask general question
- Check ticket
- Create ticket
- Update ticket
- Escalate ticket

Node 3 — Decide

Determines whether a tool is required.

Node 4 — Execute Tool

Calls the required backend tool.

Node 5 — Generate Response

Uses the LLM to generate the final response.

Node 6 — Escalation

If human support is required, the workflow can trigger escalation.

---

12. AI Response Suggestion

Support agents should be able to request an AI-generated response suggestion.

Example:

Customer message:

I've been waiting for three days and nobody has solved my issue.

The system may generate a suggested professional response.

The support agent must be able to:

- View suggestion
- Edit suggestion
- Send final response

The AI must not automatically send the response without human confirmation.

---

13. AI Ticket Classification

When a ticket is created, the AI may classify it.

The classification can include:

- Category
- Priority
- Short summary
- Suggested action

Example:

Input:

I cannot access my account after changing my password.

Possible AI output:

```
Category: Account Issue
Priority: High
Summary: Customer cannot access account after password change.
Suggested Action: Verify account authentication and password reset status.
```

The team should store the useful classification results in PostgreSQL.

---

14. n8n Automation

n8n must be integrated into the project.

n8n should handle automation instead of putting every automation directly inside FastAPI.

At least 3 meaningful workflows are required.

Workflow 1 — New Ticket Notification

Trigger:

New ticket created.

Flow:

New Ticket
↓
n8n
↓
Get Ticket Information
↓
Notify Support Team
↓
Send Email

Workflow 2 — Ticket Escalation

Trigger:

Ticket becomes Critical or is escalated by the Agent.

Flow:

Ticket Escalated
↓
n8n
↓
Notify Admin
↓
Send Email
↓
Record Escalation

Workflow 3 — Google Sheets Reporting

n8n should send selected ticket information to Google Sheets.

Example columns:

- Ticket ID
- Customer
- Category
- Priority
- Status

- Assigned Agent
- Created Date
- Resolution Date

This workflow demonstrates external-system integration.

Optional Workflow 4 — Follow-up

After a ticket is resolved:

```
Ticket Resolved
    ↓
   n8n
    ↓
Wait
    ↓
Send Customer Follow-up
```

The team can ask the customer whether the issue was successfully resolved.

---

15. FastAPI Backend

The backend must be implemented using FastAPI.

The API should be divided into logical routers.

Suggested structure:

```
app/
│
├── main.py
│
├── routers/
│   ├── auth.py
│   ├── users.py
│   ├── tickets.py
│   ├── messages.py
│   └── ai.py
│
├── models/
│   ├── user.py
│   ├── ticket.py
│   └── message.py
```

```
│
├── schemas/
│   ├── user.py
│   ├── ticket.py
│   └── message.py
│
├── services/
│   ├── auth_service.py
│   ├── ticket_service.py
│   └── ai_service.py
│
└── agent/
    ├── tools.py
    ├── graph.py
    └── prompts.py
```

The exact structure may be modified according to the team's implementation.

---

16. Frontend Requirements

The frontend must provide a usable web interface.

Minimum pages:

Public

- Login
- Registration

Customer

- Customer Dashboard
- My Tickets
- Create Ticket
- Ticket Details
- Chatbot

Support Agent

- Agent Dashboard
- Assigned Tickets
- Ticket Details
- AI Response Suggestion

Admin

- Admin Dashboard
- Users
- Agents
- Tickets
- Statistics

---

17. Dashboard Requirements

The dashboard should display meaningful statistics.

Examples:

- Total Tickets
- Open Tickets
- In Progress Tickets
- Resolved Tickets
- Critical Tickets
- Average tickets per category
- Tickets assigned to agents

Charts are optional but recommended.

---

18. API Requirements

The team must expose REST APIs.

Examples:

```
POST   /auth/register
POST   /auth/login

GET    /tickets
POST   /tickets
GET    /tickets/{ticket_id}
PUT    /tickets/{ticket_id}
DELETE /tickets/{ticket_id}

GET    /tickets/{ticket_id}/messages
POST   /tickets/{ticket_id}/messages
```

POST /ai/chat
POST /ai/suggest-response
POST /ai/classify-ticket

The final API design may differ as long as the required functionality is available.

---

19. Final Deliverables

Each team must submit:

1. GitHub repository
2. Working application
3. PostgreSQL database/schema
4. FastAPI backend
5. Frontend
6. AI chatbot
7. LangChain implementation
8. LangGraph implementation
9. AI Agent
10. Agent tools
11. n8n workflows
12. README
13. .env.example
14. Presentation
15. Final demo
16. Documentation

---

20. Final Presentation Requirements

The final presentation should explain:

1. Business Problem
2. Proposed Solution
3. System Architecture
4. Database Design
5. Backend Architecture
6. Authentication

7. AI Architecture
8. LangChain
9. LangGraph
10. Agent & Tool Calling
11. n8n Workflows
12. Git/GitHub Team Workflow
13. Live Demo
14. Challenges
15. Future Improvements
16. Documentation

The live demo must include at least one complete end-to-end scenario.

---

21. Future Enhancements

The following features are optional and should only be considered after completing all mandatory requirements:

- RAG
- Knowledge Base
- Vector Database
- Customer Satisfaction Analysis
- Voice Support
- Multi-Agent Architecture
- Advanced Analytics
- Real-time WebSocket Chat
- Deployment to Cloud
- Advanced monitoring

These are NOT required for the graduation project.
